

Tactile Hands: A Force-Sensing Robotic Hand for Safe Grasping

BRYCE HACKEL, University of California, San Diego, USA

ALI EL LAHIB, University of California, San Diego, USA

THOMAS NGHIEM, University of California, San Diego, USA

SHREE VENKATESH, University of California, San Diego, USA

SIDATH DAHAM WIJESINGHE, University of California, San Diego, USA

Robotic hands that grasp without sensing how hard they squeeze can crush fragile objects, yet no open-source hand under \$200 offers fingertip force sensing. We present Tactile Hands, a sub-\$200, four-fingered robotic hand that adds calibrated fingertip force-sensing resistors (FSRs) to the open-source AmazingHand. A custom ESP32-S3 board reads four FSRs through voltage dividers and streams calibrated forces to a ROS 2 host, where a force-aware controller halts finger closure at a safe threshold. Calibration reaches better-than-1 N accuracy. Mounted on a teleoperated arm and driven in VR, the same controller grasps a thin-walled cup without crushing it, where a force-blind grasp collapses it. Our work presents a low-cost foundation for researchers and hobbyists to perform tactile manipulation.

Additional Key Words and Phrases: tactile sensing, force-sensing resistors, robotic hand, grasping, calibration, teleoperation, ROS 2, low-cost robotics

1 Introduction

A robot arm can be precise to the millimeter and still be dangerous, because position tells it nothing about force. A hand that cannot feel how hard it is gripping will crush a paper cup as readily as it lifts a steel block. Humans avoid this with fingertip touch that modulates grip in real time; bringing the same fingertip force sensing to robots is what separates a hand that “closes until it stops” from one that “closes until it feels enough.”

High-end research hands such as the Shadow Dexterous Hand or sensorized hands built around GelSight [16] and DIGIT [6] tactile sensors demonstrate the value of touch, but they cost thousands of dollars and are out of reach for most students and small labs. Low-cost open-source hands have begun to close this gap on the mechanical side: the LEAP Hand [12] and the AmazingHand [9] are anthropomorphic, 3D-printable, and affordable. However, these affordable platforms ship without fingertip force sensing. To our knowledge, there is no open-source robotic hand available for under roughly \$200 that provides calibrated fingertip force measurement and uses it to grasp safely. This is the gap our project targets.

Why is this capability missing at low cost? High-fidelity tactile sensors are expensive (Section 2), whereas force-sensing resistors (FSRs) cost a few dollars each, need a single analog channel, and add no per-finger compute. Their drawback is signal quality: FSRs are nonlinear, hysteretic, and vary unit to unit, so a raw reading is not a force. We treated this as an engineering problem rather than a fundamental limit. With a controlled calibration procedure and a simple per-sensor model, inexpensive FSRs can be made accurate enough (better than 1 N) to close a useful safety loop, which is all that grasping a fragile object without crushing it actually requires.

We present **Tactile Hands**, a four-fingered robotic hand built on the open-source AmazingHand mechanical platform and augmented with a fingertip force-sensing subsystem of our own design. Each fingertip carries a force-sensing resistor (FSR); a custom ESP32-S3 board reads all four sensors through voltage dividers, converts raw ADC readings to Newtons using a calibrated power-law model, and streams the values over USB to a ROS 2 host. A force-aware grasp controller continuously compares each fingertip force against a safe threshold and stops closing a finger the moment that threshold is crossed, allowing the hand to wrap a thin-walled paper cup

Authors' Contact Information: Bryce Hackel, University of California, San Diego, La Jolla, California, USA, bhackel@ucsd.edu; Ali El Lahib, University of California, San Diego, La Jolla, California, USA, aellahib@ucsd.edu; Thomas Nghiem, University of California, San Diego, La Jolla, California, USA, tnghiem@ucsd.edu; Shree Venkatesh, University of California, San Diego, La Jolla, California, USA, s1venkatesh@ucsd.edu; Sidath Daham Wijesinghe, University of California, San Diego, La Jolla, California, USA, swijesinghe@ucsd.edu.

and hold it without collapsing it. The complete bill of materials remains under \$200. That figure covers the hand itself driven by a laptop webcam; the Valve Index controllers and ARCTOS arm are stretch additions on top.

Beyond the core force-sensing minimum viable product (MVP), we built three input and simulation extensions that share the same ROS 2 backbone: a webcam-based hand-pose pipeline that mirrors a human operator’s finger curl onto the hand, a physics simulation of the AmazingHand for kinematics validation and future sim-to-real work, and a virtual-reality (VR) teleoperation pipeline that streams 6-DoF controller pose into ROS 2. Most recently, we mounted the hand on an open-source ARCTOS robotic arm [2] and teleoperated its end-effector from VR controllers (Section 3.10). We further added vision-guided, multi-object grip selection: a fine-tuned YOLO26 object detector recognizes the target object in a webcam image and selects an appropriate per-object grip force automatically.

Contributions. This paper makes the following contributions:

- A **sub-\$200, open hardware design** that adds fingertip force sensing to a low-cost, 3D-printable robotic hand, including a custom ESP32-S3 sensing board with per-channel voltage dividers.
- A **repeatable FSR calibration procedure** using stacked 3D-printed reference weights and a power-law fit that maps raw ADC counts to force with better-than-1 N accuracy (max error 25.7 g $\approx$ 0.25 N).
- A **force-aware grasp controller** on a ROS 2 host that fuses the calibrated fingertip forces with finger position commands to grasp and release a fragile object without crushing it.
- A **teleoperation demonstration of the force-aware safety**: using VR control of the arm-mounted hand, force-thresholded grasping holds a fragile cup that an otherwise-identical force-blind grasp visibly collapses, showing the contribution of the tactile feedback.
- A set of **teleoperation and simulation extensions** (webcam hand-pose mirroring, physics simulation, and VR controller streaming) that share one ROS 2 interface, culminating in a VR-teleoperated, arm-mounted hand and a vision-guided, multi-object grip-selection pipeline driven by a fine-tuned YOLO26 detector.

The remainder of the paper reviews related work (Section 2), details the system design and each subsystem (Section 3), reports our milestone history including mid-quarter revisions and the problems we encountered (Section 4), and concludes with future work (Section 5).

2 Related Works

Our project sits at the intersection of low-cost robot hands, tactile sensing, and teleoperation. We summarize the most relevant prior work in each area and explain how Tactile Hands relates to it.

Low-cost and open-source robotic hands. The LEAP Hand [12] demonstrated that an anthropomorphic, dexterous hand can be built for a few hundred dollars from off-the-shelf servos and 3D-printed parts, and it has become a popular platform for robot learning. The AmazingHand [9] pushes cost down further with a fully 3D-printed, tendon- and linkage-driven design powered by small serial-bus servos; we adopt it as our mechanical base. These platforms prioritize low cost and reproducibility, but they ship without fingertip force sensing. Our work keeps their affordability while adding the missing tactile channel.

Tactile sensing technologies. Tactile sensing for robots spans a wide design space, surveyed by Dahiya et al. [3]. High-resolution vision-based sensors such as GelSight [16] and DIGIT [6] recover both contact geometry and force at the cost of an embedded camera, illumination, and significant compute per fingertip: excellent fidelity, but poorly matched to a sub-\$200 budget across four fingers. At the opposite end, force-sensing resistors (FSRs) are inexpensive piezoresistive sensors whose resistance drops nonlinearly with applied normal force [5]. FSRs are noisy, hysteretic, and require per-sensor calibration, but they cost on the order of a few dollars and need only a

single ADC channel each. We choose FSRs precisely because they make four-fingertip force sensing economically feasible, and we address their nonlinearity with an explicit calibration model.

Force-aware and safe grasping. A long line of work shows that closing the loop on contact force improves grasping robustness and prevents damage to fragile objects. Learning-based systems such as OpenAI’s in-hand manipulation work [1] achieve remarkable dexterity but rely on extensive simulation and high-end hardware. Our controller is deliberately simple, a thresholded force limit on each finger, and it shows that even a minimal force feedback loop on cheap sensors yields a meaningful safety guarantee.

Teleoperation. Vision- and VR-based teleoperation map a human operator’s motion onto a robot. DexPilot [4] and AnyTeleop [10] teleoperate dexterous hand-arm systems from camera observations of the operator’s hand. For lightweight pose capture we use Google’s MediaPipe Hands [17], which provides real-time 21-landmark hand tracking from a single webcam. For VR we stream controller pose using Valve’s OpenVR [15] interface, building on the `unity_ros_teleoperation` framework [11].

Object recognition for grip selection. Choosing how firmly to grasp an object first requires recognizing what it is. The YOLO (“You Only Look Once”) family of single-stage detectors is the standard tool for fast, real-time object detection, and the latest release, Ultralytics YOLO26 [14], is engineered for efficient, low-latency inference on edge hardware. Because the appropriate grip force depends on the object—a full bottle must be held more firmly than an empty one—we fine-tune YOLO26 on a small custom-annotated dataset of our test objects and use the predicted class to look up a per-object force threshold (Section 3.11), turning recognition into an automatic grip-force selector that feeds the existing force-aware loop.

Simulation and sim-to-real. Physics simulators such as MuJoCo [13] and NVIDIA Isaac Gym [8] are standard tools for validating robot kinematics and training control policies before deployment. Transferring such policies to hardware remains challenging due to the reality gap [18]. We bring up the AmazingHand in simulation to verify finger kinematics and to lay the groundwork for sim-to-real in-hand manipulation, while keeping all real-time control on ROS 2 [7] so that simulation, vision, and VR subsystems can be developed in parallel against the same interface.

3 Technical Material

3.1 System Architecture

The system is a sensing-and-actuation hand connected to a ROS 2 host. On the hand, an ESP32-S3 reads four FSRs through voltage dividers and streams calibrated forces over USB; on the host, a serial-bridge node publishes those forces and forwards servo commands back. Every input subsystem (hand-pose estimation, VR streaming, simulation) produces target finger or end-effector commands, and the force-aware grasp controller decides to allow commanded motion only while fingertip forces stay below a safe limit (Figure 1).

3.2 Mechanical Platform

The hand is based on the open-source AmazingHand [9], a four-fingered, fully 3D-printed design. Each finger is actuated by two Feetech SCS0009 serial-bus micro servos (eight in total), giving four independently controllable fingers. We kept the original four-finger morphology, which is sufficient for force-aware power and pinch grasps on our test objects and keeps the hand compact enough to mount on a small arm. The fingertips were modified to host an FSR pad and to route its leads back toward the palm (Figure 2, left). The entire hand, prototype board, and status LEDs are enclosed in a 3D-printed shell printed on a Bambu Lab A1 Mini, with the electronics mounted on the back of the hand (Figure 3).

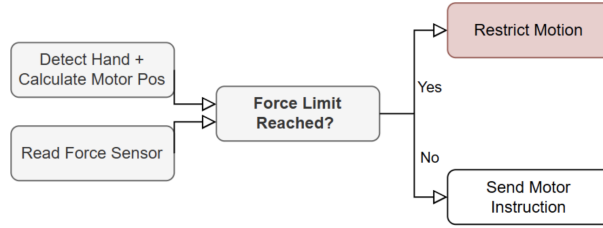


Fig. 1. Force-aware grasp control loop. Target finger positions and live FSR forces are constantly compared. If a finger’s force exceeds the threshold, motion is restricted and a red status LED lights; otherwise the servo command is issued.



Fig. 2. The Tactile Hand. Left: assembled on its base, with a force-sensing resistor bonded to each fingertip and its leads routed back toward the palm. Right: performing a force-aware grasp of a thin-walled cup, halting finger closure the moment the measured fingertip force exceeds the safe threshold (Section 3.6).

3.3 Sensing Electronics

Each fingertip carries a Pololu #2728 FSR. Because an FSR behaves as a force-dependent variable resistor, we read it with a simple voltage divider: the FSR is placed in series with a fixed reference resistor $R_{\text{ref}} = 48 \text{ k}\Omega$, and the ESP32-S3’s 12-bit ADC samples the divider’s midpoint, $V_{\text{adc}} = V_{\text{cc}} R_{\text{ref}} / (R_{\text{FSR}}(F) + R_{\text{ref}})$. As applied force F increases, R_{FSR} falls and V_{adc} rises toward V_{cc} ; the ADC reports a raw count $r \in [0, 4095]$. We use the ESP32-S3 for sending the data back to the host. The board is a protoboard hosting the ESP32-S3, the four voltage dividers, and vertical 0.1-inch (2.54 mm) headers for the FSR Dupont leads. Power comes from a 2S LiPo (7.4 V nominal) stepped down to 5 V by a buck converter.

3.4 Firmware and ROS 2 Data Pipeline

The firmware is built on the Arduino-ESP32 core. It samples the four ADC channels at 100 Hz, applies the per-channel calibration described in Section 3.5 to convert raw counts to Newtons, and emits a timestamped CSV line (timestamp followed by four force values) over a USB CDC serial link at 115200 baud.

On the host, a ROS 2 (Humble, on Ubuntu) serial-bridge node parses the CSV stream and publishes the calibrated forces on `/tactile/forces`. The same node subscribes to `/servo/commands` and forwards target servo positions to the ESP32 over the same serial link.

3.5 Force Sensor Calibration

FSRs are strongly nonlinear and vary unit to unit, so raw ADC counts must be mapped to physical force. We built a calibration rig that holds a sensor flat and level, then stacked 3D-printed PLA reference weights on a loading

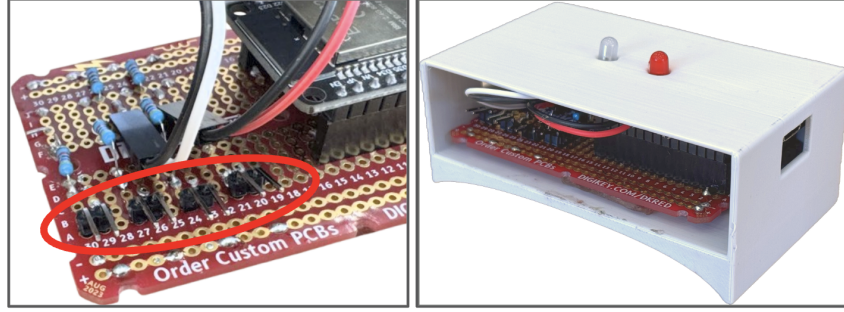


Fig. 3. Custom ESP32-S3 sensing board (left) with the four FSR voltage-divider channels highlighted, and the 3D-printed enclosure with status LEDs mounted on the back of the hand (right).

plate so a known mass rests on the active area (Figure 4, left). For each load we logged the ADC reading over serial. Sweeping a range of loads produced a set of (raw count, force) pairs.

The FSR’s count-to-force relationship is well captured by a power law,

$$F(r) = ar^b, \quad (1)$$

which we fit by nonlinear least squares. Across seven loading points the fit yielded $a = 1.07 \times 10^{-9}$ and $b = 3.16$, with a root-mean-square error of 17.1 g and a maximum error of 25.7 g (Figure 4, right). Since $1 \text{ N} \approx 102 \text{ g-force}$, a 25.7 g worst case corresponds to roughly 0.25 N—comfortably within our 1 N accuracy target. The procedure is documented and repeatable.

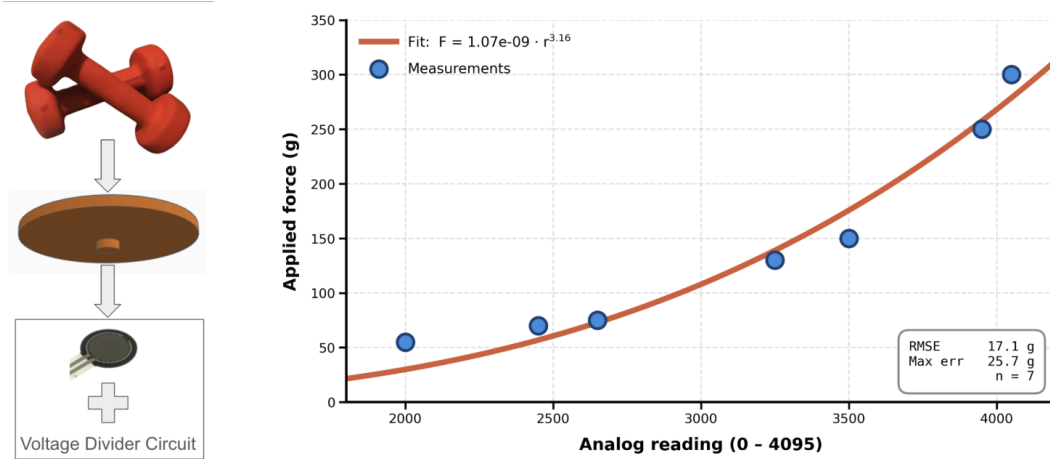


Fig. 4. Force-sensor calibration. Left: the calibration rig, a 3D-printed loading plate on the FSR’s active area loaded with stacked PLA reference weights to apply a known mass. Right: a power-law model fit to seven stacked-weight loading points maps raw ADC counts to applied force, with worst-case error 25.7 g ($\approx 0.25 \text{ N}$), well within the 1 N target.

3.6 Force-Aware Grasp Control

The grasp controller (Figure 1) runs on the host at the sensor rate. A finger is driven toward a target curl by an open-loop closing trajectory (during MVP demos) or by a teleoperation input. At every cycle the controller

reads that finger’s calibrated force from `/tactile/forces`. While the force is below the safe threshold τ , the commanded position is forwarded to the servo and the finger continues to close. The instant the force crosses τ , the controller freezes that finger’s target (the “restrict motion” branch) so it stops squeezing harder even though the nominal trajectory would keep closing, and a red status LED (one of two indicator LEDs on the board) lights to signal that the threshold has been reached. Release is triggered on demand by commanding the fingers open. In practice this lets the hand wrap a thin-walled paper cup and halt at its safe limit without buckling the wall (Figure 2, right).

3.7 Vision-Based Hand-Pose Estimation

To drive the hand from a human gesture we built a webcam pipeline on MediaPipe Hands [17] and OpenCV, which returns 21 hand landmarks in real time. From the landmarks we estimate each finger’s joint angles, reduce them to a single normalized curl value in $[0, 1]$ per finger, and map that value onto the servo’s angular range. The result is a four-element command vector published on `/servo/commands`, so the robotic hand mirrors the operator’s finger curl live (Figure 5). Because the pipeline outputs the same normalized curl interface used elsewhere, it is a drop-in alternative to the VR input.

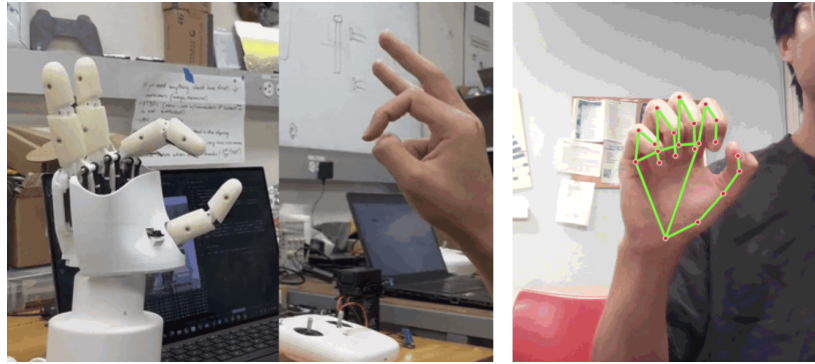


Fig. 5. Hand-pose mirroring. MediaPipe-based finger-pose estimation from a webcam (right) is mapped to per-finger servo commands so the robotic hand reproduces the operator’s gesture (left).

3.8 Physics Simulation

We brought up the AmazingHand in the MuJoCo physics simulator [13] by loading its URDF/MJCF model and verifying that the simulated finger kinematics and curl poses match the real hand. The simulation serves two purposes: it is a safe sandbox for validating control logic before running it on hardware, and it is the foundation for sim-to-real experiments. The main one is an in-hand cube-rotation policy trained in simulation that we intend to transfer to the real hand (Figure 6, left), where the reality gap [18] between simulated and physical FSR contact remains the central challenge.

3.9 VR Teleoperation and Arm Integration

Our final input modality is VR teleoperation. We initially built a Meta Quest 2 APK in Unity 2022.3 LTS from the `unity_ros_teleoperation` framework [11] and successfully streamed 6-DoF controller pose (position and orientation) into ROS 2 on Ubuntu. While functional, the Quest 2 path was awkward to integrate with our Python-based control stack. We therefore switched to Valve Index controllers, read through OpenVR/pyopenvr [15], which gave us a clean pure-Python interface and native per-finger capacitive tracking. With the Index controllers,

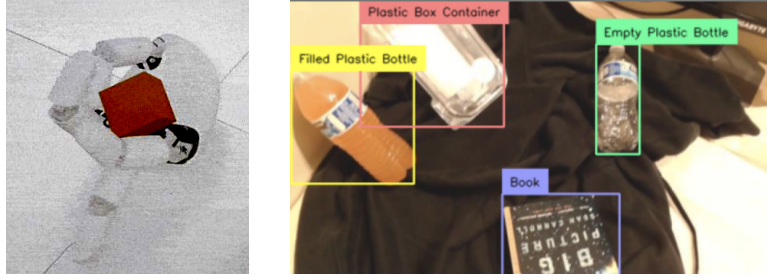


Fig. 6. Left: simulated in-hand manipulation. The AmazingHand model grasps a cube in MuJoCo, the environment used to develop the in-hand cube-rotation policy we intend to transfer to hardware. Right: vision-guided grip selection. A fine-tuned YOLO26 detector recognizes each object and its class (e.g., filled vs. empty plastic bottle), and the detected class selects a per-object grip-force threshold the force-aware loop enforces (Section 3.11).

controller pose is published as a ROS 2 topic at interactive rates. The same per-finger capacitive sensing lets the operator’s finger motion drive the hand’s fingers directly (Figure 7, left), so a single controller supplies both the 6-DoF wrist pose and a per-finger curl signal on the shared `/servo/commands` interface.



Fig. 7. VR teleoperation. Left: the Valve Index controller’s native capacitive finger tracking is streamed over ROS 2 to drive the robotic hand’s finger curl directly from the operator’s hand. Right: the controller’s 6-DoF pose drives the ARCTOS arm end-effector through our framework (frame calibration, IK, and motion smoothing), giving VR control of the arm-mounted hand.

To turn streamed pose into motion we mounted the hand on an open-source ARCTOS robotic arm [2] and mapped controller pose to the arm’s end-effector. In the integrated platform the Tactile Hand is fixed to the ARCTOS end-effector, and the whole arm-hand chain is driven from the VR controller through the same ROS 2 stack used for the standalone hand. We built a full end-to-end teleoperation framework for the arm: moving an Index controller moves the ARCTOS arm’s end-effector through our ROS 2 pipeline, with controller-to-arm frame calibration, inverse kinematics, and motion smoothing all in place, so the full chain from VR input to physical arm motion is closed (Figure 7, right).

3.10 Teleoperated Force-Aware Grasping

Mounting the hand on the arm let us ask a sharper question than the scripted MVP demo could: does the force-aware safety guarantee survive once a human is in the loop and the grasp is no longer a fixed trajectory? To test this we kept the force-aware controller of Section 3.6 active on the arm-mounted hand while an operator teleoperated the arm to a fixed, thin-walled 3D-printed cup and triggered a grasp. We ran the same teleoperated grasp under two conditions: once with the force-aware controller enabled, and once with force feedback disabled

so the fingers simply close to a fixed curl; the operator drives the same Index controller to the same cup in both runs.

The contrast is stark and repeatable. With force-thresholding enabled, each finger halts at the cup’s safe limit and the hand holds the cup with its wall intact; with feedback disabled, the identical teleoperated motion drives the fingers past the safe force and visibly buckles the cup wall (Figure 8). Because nothing changes between the two runs except the force loop, the comparison isolates the contribution of the tactile channel: the outcome flips from a clean hold to a crushed cup. The same safety behavior that protected the cup in the scripted MVP holds up when a human is driving the arm in VR.

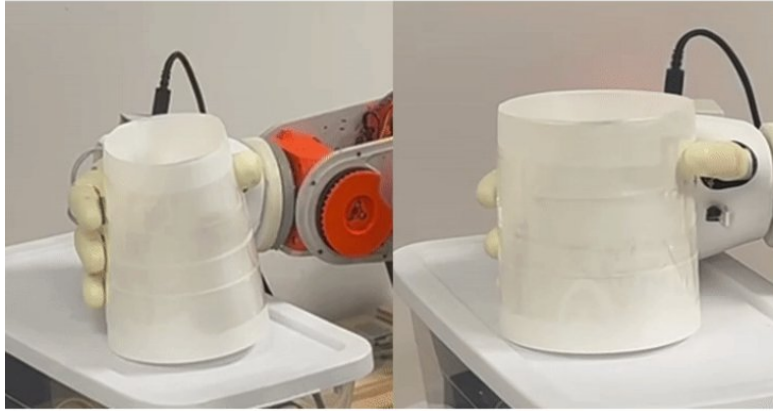


Fig. 8. Teleoperated force-aware grasping. Under the same VR-teleoperated motion, the force-blind grasp (left) crushes the thin-walled cup, while the force-threshold grasp (right) holds it with the wall intact. Only the force loop differs between the two runs.

3.11 Multi-Object Force-Awareness and Vision-Guided Grip Selection

A single hand-tuned threshold is enough for one known object, but a genuinely useful hand must choose an appropriate grip force for whatever it is handed. We extended the force-aware controller toward automatic, per-object grip selection and exercised it on a small set of everyday objects that demand different grips. The hardest case is an empty versus a filled plastic bottle, which look nearly identical but differ sharply in weight and therefore in the grip force needed to hold them without slipping or crushing.

Our pipeline recognizes the target in a webcam frame with a fine-tuned YOLO26 object detector [14]. We annotated a small custom dataset of our objects (e.g., filled plastic bottle, empty plastic bottle, plastic box container, and book) and fine-tuned YOLO26 on it, giving real-time bounding-box detections with a class label per object (Figure 6, right). Each class maps to a hand-tuned target grip force, with heavier or sturdier objects getting a firmer threshold and lighter or more fragile ones a gentler threshold, and the detected class selects which threshold the force-aware loop enforces. The detector changes only the source of the target force; the downstream /tactile/forces feedback path and the restrict-motion logic of Section 3.6 are reused unchanged, which kept the addition lightweight.

This experiment also exposes a calibration limitation: FSRs respond differently to off-axis and shear loading than to the normal loading used during calibration (Section 3.5), so a misaligned grip reads a biased force. We have begun characterizing each sensor’s off-axis response and feeding it back into the physics simulation (Section 3.8) to tighten the sim-to-real match.

Table 1. Milestone status. All MVP-track items are complete; reach goals range from complete to not completed, consistent with their deferred priority.

Milestone	Target	Status
System design, board fabrication & components	W2–W4	Complete
Hand assembly, FSR integration & firmware	W3–W5	Complete
FSR calibration to within 1 N	W4–W6	Complete
Simulation bring-up of AmazingHand	W3–W5	Complete
Force-aware grasping closed loop	W5–W6	Complete
MVP completion & validation	W6	Complete
Hand-pose estimation pipeline (reach)	W7–W9	Complete
VR teleoperation: pose streaming, arm integration & force-aware grasp (reach)	W7–W9	Complete
Multi-object force awareness (reach)	W8–W9	Complete
Sim-to-real in-hand rotation (reach)	W8–W9	Not completed
Final paper, demo video, GitHub repo	W9–W10	In progress

4 Milestones

We targeted MVP completion by Week 6, with reach goals in Weeks 7–10. Table 1 lists each milestone and its final status.

4.1 Mid-Quarter Revisions

Our main mid-quarter revision was the VR hardware: we changed from the chartered Meta Quest 2 to Valve Index controllers (Section 3.9). We had a working Quest 2 APK, but it was awkward to drive from our Python stack.

4.2 What We Did and Did Not Finish

All MVP deliverables were met, and the reach-goal outcomes are listed in Table 1. Two items remain unfinished: the sim-to-real transfer of the in-hand rotation policy (trained in simulation but not yet moved to hardware), and finger adduction/abduction control (our curl interface captures flexion only). Because reach goals were started only after the Week 6 MVP, these gaps reflect the time available afterward, not a failure of the core deliverable.

4.3 Problems and Mitigations

FSR nonlinearity and unit-to-unit variation threatened our 1 N target; the stacked-weight calibration rig and power-law model brought worst-case error to 0.25 N. Quest 2 integration friction was resolved by switching to Valve Index. Mounting the hand on the arm surfaced three further issues: ARCTOS wiring needed rework before the arm drove reliably; off-axis fingertip loading during arm-driven grasps biases readings relative to the normal-load calibration (we are characterizing this and feeding it back into simulation, Section 3.11); and the read–decide–actuate loop has enough latency that the operator must close slowly to avoid overshooting the threshold. The hand’s wide palm limited its reachability and made small objects hard to grasp, so we favored larger test objects.

5 Conclusion

A robot hand that cannot feel how hard it is gripping is dangerous, and that safety feature has been missing from affordable open-source hands. We presented Tactile Hands, a sub-\$200 four-fingered robotic hand that adds calibrated fingertip force sensing to the open AmazingHand platform and uses it to grasp a fragile object without

crushing it. Mounted on a VR-teleoperated arm, it delivered our central result: under identical teleoperated motion, the force-aware grasp holds a thin-walled cup while a force-blind grasp crushes it.

The significance of the work is accessibility: the most safety-critical sensing capability of a robot hand, knowing its own grip force, can be added to a cheap, reproducible platform without specialized hardware, and keeps working once the hand is on a human-driven arm, lowering the barrier to safe manipulation research and education.

Future work follows from our partial milestones: complete per-finger calibration, grow the YOLO26 dataset toward learned weight-aware force targets, carry the off-axis system-identification results into the simulator to close the sim-to-real gap for the in-hand rotation policy, and refine the VR arm teleoperation into reliable pick-and-place. All hardware designs, firmware, calibration data, and software will be released publicly so that others can reproduce a low-cost, force-sensing dexterous hand.

References

- [1] Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, et al. 2020. Learning Dexterous In-Hand Manipulation. *The International Journal of Robotics Research* 39, 1 (2020), 3–20.
- [2] Arctos Robotics. 2024. Arctos Robotics: Open-Source 3D-Printable Robotic Arm. <https://arctosrobotics.com>. Accessed 2026.
- [3] Ravinder S. Dahiya, Giorgio Metta, Maurizio Valle, and Giulio Sandini. 2010. Tactile Sensing—From Humans to Humanoids. *IEEE Transactions on Robotics* 26, 1 (2010), 1–20.
- [4] Ankur Handa, Karl Van Wyk, Wei Yang, Jacky Liang, Yu-Wei Chao, Qian Wan, Stan Birchfield, Nathan Ratliff, and Dieter Fox. 2020. DexPilot: Vision-Based Teleoperation of Dexterous Robotic Hand-Arm System. In *IEEE International Conference on Robotics and Automation (ICRA)*. 9164–9170.
- [5] Interlink Electronics. 2024. FSR Integration Guide and Evaluation Parts Catalog. <https://www.interlinkelectronics.com>. Force-Sensing Resistor application notes.
- [6] Mike Lambeta, Po-Wei Chou, Stephen Tian, Brian Yang, Benjamin Maloon, Victoria Rose Most, Dave Stroud, Raymond Santos, Ahmad Byagowi, Gregg Kammerer, Dinesh Jayaraman, and Roberto Calandra. 2020. DIGIT: A Novel Design for a Low-Cost Compact High-Resolution Tactile Sensor with Application to In-Hand Manipulation. *IEEE Robotics and Automation Letters* 5, 3 (2020), 3838–3845.
- [7] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. 2022. Robot Operating System 2: Design, Architecture, and Uses in the Wild. *Science Robotics* 7, 66 (2022).
- [8] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. 2021. Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning. [arXiv:2108.10470](https://arxiv.org/abs/2108.10470)
- [9] Pollen Robotics. 2025. Amazing Hand: An Open-Source, Low-Cost, 3D-Printed Robotic Hand. <https://github.com/pollen-robotics/AmazingHand>. Accessed 2026.
- [10] Yuzhe Qin, Wei Yang, Binghao Huang, Karl Van Wyk, Hao Su, Xiaolong Wang, Yu-Wei Chao, and Dieter Fox. 2023. AnyTeleop: A General Vision-Based Dexterous Robot Arm-Hand Teleoperation System. In *Proceedings of Robotics: Science and Systems (RSS)*.
- [11] Robotic Systems Lab, ETH Zurich. 2023. unity_ros_teleoperation. https://github.com/leggedrobotics/unity_ros_teleoperation. Accessed 2026.
- [12] Kenneth Shaw, Ananye Agarwal, and Deepak Pathak. 2023. LEAP Hand: Low-Cost, Efficient, and Anthropomorphic Hand for Robot Learning. In *Proceedings of Robotics: Science and Systems (RSS)*.
- [13] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A Physics Engine for Model-Based Control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 5026–5033.
- [14] Ultralytics. 2026. Ultralytics YOLO26. <https://docs.ultralytics.com/models/yolo26>. Open-source real-time object detection models. Accessed 2026.
- [15] Valve Corporation. 2024. OpenVR SDK. <https://github.com/ValveSoftware/openvr>. Accessed 2026.
- [16] Wenzhen Yuan, Siyuan Dong, and Edward H. Adelson. 2017. GelSight: High-Resolution Robot Tactile Sensors for Estimating Geometry and Force. *Sensors* 17, 12 (2017), 2762.
- [17] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*.
- [18] Wenshuai Zhao, Jorge Peña Queraltá, and Tomi Westerlund. 2020. Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey. *IEEE Symposium Series on Computational Intelligence (SSCI)* (2020), 737–744.